# Experiment 1: Perceptron with Fixed Increment Learning Algorithm (AND Function)

## Question

Implement a program for a perceptron with a fixed increment learning algorithm. The program uses the logical AND function as an example. The perceptron is trained until no change in weight is required.

## Theory

The perceptron is one of the simplest types of artificial neural networks (ANNs) that can be used for binary classification tasks. It consists of:

- Input layer (features of the dataset).
- Weights (adjusted iteratively to minimize classification error).
- Activation function (Binary step function).
- Output layer (predicted class label).

A perceptron learns by adjusting its weights based on misclassified examples using a fixed increment method. The learning process continues until the perceptron classifies all training data correctly or reaches a predefined iteration limit.

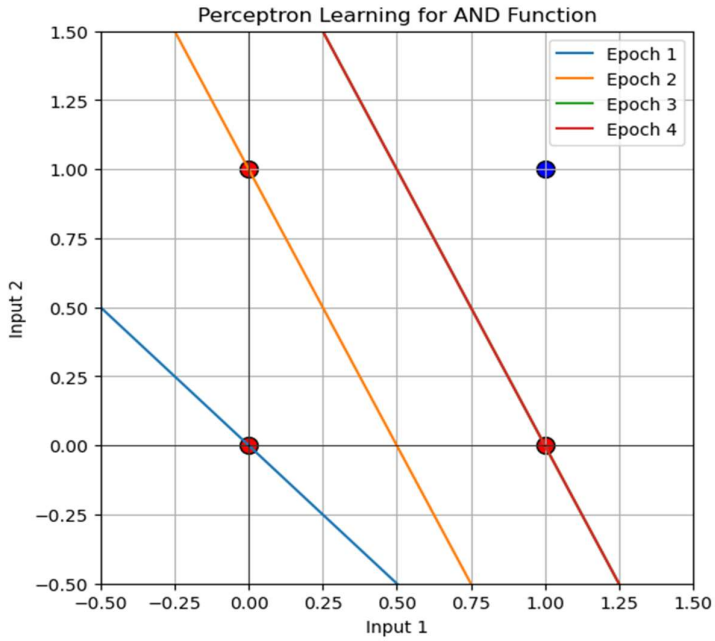
For an input vector $X = [x_1, x_2]$, weights $W = [w_1, w_2]$, and bias $b$, the perceptron computes: $y = f(WX + b)$ where $f$ is the activation function defined as: $f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$

## Implementation:

```
import numpy as np
import matplotlib.pyplot as plt

# AND gate input and output
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1]) # Expected output

# Initialize weights and bias to zero
weights = np.zeros(2)
bias = 0
learning_rate = 0.1

# Function to predict output
def predict(inputs):
    summation = np.dot(inputs, weights) + bias
    return 1 if summation >= 0 else 0
```

```python
# Training the perceptron until convergence
converged = False
iterations = 0
decision_boundaries = []

while not converged:
    iterations += 1
    previous_weights = weights.copy()
    previous_bias = bias
    for i in range(len(X)):
        prediction = predict(X[i])
        error = y[i] - prediction
        weights += learning_rate * error * X[i]
        bias += learning_rate * error

    # Store decision boundary after each epoch
    if weights[1] != 0:
        x_vals = np.linspace(-0.5, 1.5, 100)
        y_vals = -(weights[0] * x_vals + bias) / weights[1]
        decision_boundaries.append((x_vals, y_vals))

    # Check for convergence
    if np.array_equal(previous_weights, weights) and previous_bias == bias:
        converged = True

# Plot decision boundary evolution
plt.figure(figsize=(6, 6))
for i, (x_vals, y_vals) in enumerate(decision_boundaries):
    plt.plot(x_vals, y_vals, label=f'Epoch {i+1}')

# Plot data points
for i in range(len(X)):
    plt.scatter(X[i][0], X[i][1], c='red' if y[i] == 0 else 'blue', marker='o', edgecolors='black', s=100)
plt.xlabel('Input 1')
plt.ylabel('Input 2')
plt.title('Perceptron Learning for AND Function')
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend()
plt.grid(True)
plt.show()

# Display final weights and number of iterations
print(f'Final Weights: {weights}')
print(f'Final Bias: {bias}')
print(f'Number of Iterations Until Convergence: {iterations}')
```

## **Output**




```
Final Weights: [0.2 0.1]
Final Bias: -0.20000000000000004
Number of Iterations Until Convergence: 4
```